JavaScript Practice: Arrays, Objects, Strings & Interview Programs

Plain JavaScript practice files. Each file **checks its own answers** when you run it: a ✓ means the example in the comment is proven correct.

> Part of the javascript-practice repository. Only Node.js is needed; there are no packages to install.

Contents

1. Files at a glance
2. How to run
3. array-methods-cheatsheet.js
4. array-methods-exercises.js / solutions.js
5. arrays-and-objects-cheatsheet.js
6. arrays-and-objects-exercises.js / solutions.js
7. js-interview-programs.js
8. string-methods-cheatsheet.js
9. string-programs.js
10. Quick reference tables
11. Suggested study plan
12. Troubleshooting

---

1. Files at a glance

#	File	Type	What's inside	Status when run
1	`array-methods-cheatsheet.js`	Cheat sheet	forEach, map, filter, reduce, some, every, slice, splice, shift, unshift	90 checks ✓
2	`array-methods-exercises.js`	Exercises	20 exercises on the methods above	0/20 until you solve them
3	`array-methods-solutions.js`	Solutions	Answers to file 2, with explanations	20/20 ✓
4	`arrays-and-objects-cheatsheet.js`	Cheat sheet	Other array methods, all object operations, arrays of objects	165 checks ✓
5	`arrays-and-objects-exercises.js`	Exercises	22 exercises on arrays & objects	0/22 until you solve them
6	`arrays-and-objects-solutions.js`	Solutions	Answers to file 5, with explanations	22/22 ✓
7	`js-interview-programs.js`	Interview programs	36 classic programs with "Remember" and "Explain" notes	87 checks ✓
8	`string-methods-cheatsheet.js`	Cheat sheet	Every built-in string method: arguments, returns, traps	116 checks ✓
9	`string-programs.js`	Interview programs	32 string programs, each solved with built-ins AND with plain loops	55 checks ✓

#	File	Type	What's inside	Status when run
10	`README.md`	This guide	What each file covers and how to use it	n/a

Recommended order: 1 → 2 → 4 → 5 → 8 → 9 → 7. Open the solutions (3, 6) only after trying the exercises.

---

2. How to run

You only need **Node.js** (any recent version; Node 20+ for `toSorted`, `toSpliced`, `with`; Node 21+ for `Object.groupBy`).

Open a terminal in VS Code (**Terminal** → **New Terminal**), then go into this folder:

```
cd practice
```

Run any file with `node`:

```
node array-methods-cheatsheet.js
node array-methods-exercises.js
node arrays-and-objects-cheatsheet.js
node arrays-and-objects-exercises.js
node js-interview-programs.js
node string-methods-cheatsheet.js
node string-programs.js
```

Reading the output

Symbol	Meaning
✓	Correct
✗	Wrong: shows `expected` and `got` so you can compare
…	Exercise not done yet (still `return todo();`)

VS Code tips

- Hover over any method (for example `.reduce`) to see its official signature. Optional parameters are shown with `?`.
- Install the **Code Runner** extension to run the open file with `Ctrl + Alt + N`.
- Use the **Outline** view (Explorer sidebar) to jump between sections.

---

3. `array-methods-cheatsheet.js`

Purpose: master the 10 array methods people confuse most: what each takes, what it returns, and whether it changes the original array.

Section	Topic	Key points
Header	Quick table	Arguments (mandatory vs optional), return value, mutates or not, memory tricks
0	8 ways to write a callback	`n => n * 2`, `(n) => {...}`, returning objects `({...})`, `function`, passing a named function, destructuring, using index/array. Traps: forgotten `return`, `map(parseInt)`
1	`forEach`	Returns `undefined`; `thisArg`; can't `break`; doesn't wait for async
2	`map`	New array, same length; transforms each item
3	`filter`	New array of kept items; `filter(Boolean)`; unique via `indexOf` / `Set`

Section	Topic	Key points
4	`reduce`	With vs without initial value; empty array error; sum, max, count-by, lookup, flatten, `reduceRight`
5	`some`	"Any?"; stops early; empty array → `false`
6	`every`	"All?"; stops early; empty array → `true`
7	`slice`	Copy a part; end excluded; negative indexes; works on strings
8	`splice`	Remove / insert / replace **in place**; returns removed items; `toSpliced`
9	`shift`	Remove first item; queue processing
10	`unshift`	Add at start; returns new **length**; `push` / pop / `shift` / `unshift` side by side
11	Chaining	`filter → map → filter`, top-N with `slice().sort().slice()`

Memory tricks from this file

- *"slice is safe, sPLICE sPLITS the original"*
- *shift/unshift = START, pop/push = END*
- *map = transform, filter = keep, reduce = boil down, forEach = just do, some = any?, every = all?*

---

4. `array-methods-exercises.js` / `array-methods-solutions.js`

Purpose: practise the methods from file 1.

How: replace each `return todo();` with your answer and run the file. Compare with the solutions file afterwards.

#	Exercise	Method
1	Total duration	forEach
2–4	Upper-case names, numbered list, map to objects	map
5–6	Failed tests, tests > 2000 ms	filter
7–9	Sum of non-skipped, group by status, slowest test	reduce
10–12	Any slow test? All names valid? `every` on empty array	some / every
13–14	First two, last item	slice
15–16	Remove an item, insert after another	splice
17–18	Process a queue, add at the start	shift / unshift
19–20	Predict the output	map without return, slice indexes

---

5. `arrays-and-objects-cheatsheet.js`

Purpose: everyday operations on arrays and objects, plus real-world patterns for arrays of objects (API responses, test results).

Part A: Arrays

Section	Topic
A1	Create & check: `[]`, `Array.of`, `Array.from`, `Array.isArray`, why `new Array(3)` is a trap
A2	Read: `arr[i]`, `at(-1)`, `indexOf`, `includes`
A3	`find`, `findIndex`, `findLast`, `findLastIndex` (find = one item, filter = array)
A4	`push`, `pop` (the END of the array)

Section	Topic
A5	`join`, `split`, `concat`, spread `[...a, ...b]`
A6	`sort` (why 10 comes before 2), compare functions, `toSorted`, `toReversed`, `with`
A7	`flat`, `flatMap`
A8	`fill`, `keys()`, `entries()`
A9	Destructuring: skip, `...rest`, defaults, swap
A10	Copy & compare: `=` doesn't copy, `[1] === [1]` is false

Part B: Objects

Section	Topic
B1	Create: literal, shorthand, computed keys, method shorthand, `fromEntries`
B2	Read: dot vs brackets, `?.` optional chaining, `??` vs `||`
B3	Add, update, delete, and the no-mutation versions with spread/rest
B4	Does a key exist? `in` vs `Object.hasOwn`
B5	`Object.keys` / `values` / `entries` / `fromEntries`, filter or map an object
B6	Looping: `for...of Object.entries` (recommended) vs `for...in`
B7	Merge: spread, `Object.assign`, shallow-merge trap with nested objects
B8	Destructuring: rename, default, nested, rest, in function parameters
B9	Copy: shallow vs deep, `structuredClone`, what a JSON copy loses
B10	Compare, `JSON.stringify` / `parse`, `Object.freeze`
B11	`Map` and `Set`: when to use them

Part C: Arrays of objects

Section	Topic
C1	Find by id, pluck a field, count, any match
C2	Update / add / remove **without mutating**
C3	Sort by a number or text field (copy first!)
C4	Group by, lookup by id, unique values, unique by field
C5	Totals, counts per status, max item, average
C6	Reshape: pick fields, drop a field, object → array

6. `arrays-and-objects-exercises.js` / `arrays-and-objects-solutions.js`

#	Exercise	Topic
1–8	`at(-1)`, find by id, findIndex, sort without mutating, join, flat, `Array.from`, array destructuring	Arrays
9–16	`?.` and `??`, count keys, filter an object, merge, immutable update, remove a key, deep copy, rename + default	Objects
17–20	Update in an array, group by, sort by field, lookup by id	Arrays of objects

#	Exercise	Topic
21–22	Predict: `[10, 9, 1, 2].sort()`, two variables pointing at one array	Traps

7. `js-interview-programs.js`

Purpose: 36 classic interview programs, solved with the methods above, written to be **easy to remember and explain**.

How to answer any coding question: R-A-C-E-S

Step	What to do
Restate	Repeat the question with an example; ask about empty input
Approach	Say the idea in one sentence before typing
Code	Write the short method version; offer the loop version if asked
Edge cases	Empty, one item, duplicates, negatives, upper/lower case
Speed	Time and space complexity

How each program is written

```
/**
 * Q        the question with an example
 * Remember one-line hook to recall the approach
 * Explain  what to SAY in the interview
 * Speed    time / space complexity
 * Edge     edge cases to mention
 */
```

Program index with memory hooks

Strings

#	Program	Remember
1	Reverse a string	SPLIT → REVERSE → JOIN
2	Palindrome	CLEAN → REVERSE → COMPARE
3	Count vowels	FILTER the vowels, take the LENGTH
4	Character frequency	REDUCE: `acc[ch] = (acc[ch] || 0) + 1`
5	First non-repeating char	COUNT first, then FIND count 1
6	Reverse words / each word	SPLIT → REVERSE → JOIN / SPLIT → MAP(reverse) → JOIN
7	Capitalize each word	First letter UP + rest down
8	Anagram	NORMALISE → SORT letters → COMPARE
9	Longest word	REDUCE keeping the longer one
10	Remove duplicate chars	`[...new Set(s)].join('')`
11	Count words	TRIM → SPLIT `/\s+/` → FILTER → LENGTH

Numbers & arrays

#	Program	Remember
12	Sum & average	REDUCE with 0; sum / length
13	Max & min	`Math.max(...arr)` or REDUCE
14	Second largest	UNIQUE → SORT desc → `[1]`, or one pass
15	Remove duplicates	`[...new Set(arr)]`
16	Find duplicates	indexOf ≠ index, or a "seen" Set
17	Missing number	EXPECTED sum − ACTUAL sum, `n(n+1)/2`
18	Two sum	MAP value → index; look up `target - n`
19	Flatten	`flat(Infinity)` or RECURSION
20	Chunk	`Array.from` CEIL(n/size) + SLICE
21	Rotate	LAST k + REST, `k % length`
22	Merge sorted arrays	TWO POINTERS
23	Intersection / difference / union	SET + FILTER by `has`
24	Move zeros	NON-zeros + ZEROS
25	Most frequent	COUNT → ENTRIES → biggest
26	FizzBuzz	Check 15 FIRST, then 3, then 5
27	Factorial	`n × f(n − 1)`, STOP at 1
28	Fibonacci	Each = previous TWO added
29	Prime numbers	Divide only up to √n
30	Sum of digits	NUMBER → STRING → SPLIT → REDUCE

Objects

#	Program	Remember
31	Group by	`(acc[key] ||= []).push(item)`
32	Count by	`acc[key] = (acc[key] || 0) + 1`
33	Invert object	ENTRIES → MAP swap → FROM ENTRIES
34	Sort objects by key	COPY, then `a - b` / `localeCompare`
35	Deep clone	`structuredClone` or RECURSION
36	Deep equal	Same primitive? Same keys? RECURSE

8. `string-methods-cheatsheet.js`

Purpose: every built-in string method: what it takes (mandatory vs optional), what it returns, and the traps.

Golden rule: strings are **immutable**. No method changes the original; always use the returned value (`s = s.trim()`).

Section	Topic	Key points
Header	Quick table	All methods with their optional arguments and return values
1	Create & convert	Quotes, template literals `${}`, `String()`, `toString(radix)`, `Number` vs `parseInt` / `parseFloat`, the `'5' + 3` vs `'5' - 3` trap, `[...s]`
2	Length & reading	`length`, `s[i]`, `at(-1)`, `charAt` (never undefined), `charCodeAt`, `String.fromCharCode`, immutability, emoji length

Section	Topic	Key points
3	Search	`indexOf` / `lastIndexOf` (with `fromIndex`), `includes`, `startsWith`, `endsWith` (with position/length), case-sensitivity, `search`
4	Extract	`slice` (negatives ok) vs `substring` (swaps, no negatives); `substr` is deprecated
5	Case, trim, pad, repeat	`toUpperCase` / `toLowerCase`, `trim` / `trimStart` / `trimEnd`, `padStart` / `padEnd` (leading zeros, masking), `repeat`
6	split & join	`split(sep, limit)`, `split('')`, the double-space trap, `split(/\s+/)`, `''.split(',')` → `['']`
7	Join strings	`+`, `concat`, template literals, `join`
8	replace & replaceAll	First match only vs all, regex `/g` and `/gi`, replacement functions, capture groups `$1`
9	Regex matching	`match` (null when nothing matches!), `matchAll` with groups, `regex.test`
10	Compare & sort	`===`, `<` / `>` and the upper-case trap, `localeCompare` (with `numeric` and `sensitivity` options)
11	Loop over a string	`for`, `for...of`, spread + array methods, `Array.from(s, fn)`
12	Character checks	`isUpper`, `isLower`, `isDigit`, `isLetter`, `isVowel`, blank check

Memory tricks

- *"Strings never change, they only return."*
- *slice takes negatives, substring doesn't.*
- *replace = first only; replaceAll or /g = every match.*
- *split = string → array; join = array → string.*

---

9. `string-programs.js`

Purpose: 32 string interview programs, each written **two ways**:

- **A) Built-in:** short, using string & array methods (`reverseA`, `palindromeA`, ...)
- **B) Loop:** without built-in helpers, for when the interviewer says "*don't use reverse/split*" (`reverseB`, `palindromeB`, ...)

Each program lists **Q**, **Remember** (built-in hook), **Loop idea**, **Explain** and **Speed**.

#	Program	Remember (built-in)	Loop idea
1	Reverse a string	SPLIT → REVERSE → JOIN	Walk from the last index
2	Palindrome	CLEAN → compare with REVERSE	Two pointers from both ends
3	Reverse word order	TRIM → SPLIT `/\s+/` → REVERSE → JOIN	Collect words, add in reverse
4	Reverse each word	SPLIT → MAP(reverse) → JOIN	Build each word backwards
5	Count words	TRIM → SPLIT → FILTER → LENGTH	Count word starts
6	Longest word	SPLIT → REDUCE keep longer	Track current & best
7	Find a substring	`includes` / `indexOf`	Compare char by char at each position
8	Check rotation	`(a + a).includes(b)`	Try every rotation
9	Vowels & consonants	`match(/[aeiou]/gi)`	Letter? vowel or consonant
10	Upper / lower / digits	One regex per group	Compare char ranges

#	Program	Remember (built-in)	Loop idea
11	Character frequency	REDUCE into an object	for...of + object
12	Max occurring char	FREQUENCY → ENTRIES → REDUCE	Loop over counts
13	First non-repeating	`indexOf === lastIndexOf`	Count, then find count 1
14	First repeating	FIND where `indexOf(ch) < i`	"Seen" store
15	Count a substring	`split(sub).length - 1`	Scan and skip past matches
16	Duplicate words	SPLIT → FILTER(indexOf ≠ i) → SET	Count words
17	Capitalize each word	`charAt(0)` UP + `slice(1)` down	Upper after a space
18	Toggle case	MAP: upper ↔ lower	Character code ± 32
19	Remove spaces	`replace(/\s/g, '')`	Keep non-spaces
20	Remove vowels	`replace(/[aeiou]/gi, '')`	Keep non-vowels
21	Remove duplicate chars	`[...new Set(s)].join('')`	Add if not already in result
22	Replace all	`replaceAll` / `split().join()`	Scan, replace, jump
23	camelCase ↔ snake_case	`replace` with a function	Capital → `_` + lower
24	Truncate	`slice(0, max) + '...'`	Copy first max chars
25	Mask	`slice(-4).padStart(len, '*')`	`*` before the last 4
26	Anagram	SORT letters and compare	Count +1 / −1
27	Pangram	SET of letters has size 26	26-slot "seen" array
28	Only digits?	`/^\d+$/.test(s)`	Every char `'0'` – `'9'`
29	Compression `aaab` → `a3b1`	regex runs → `char + length`	Count same neighbours
30	Longest common prefix	Shrink until `every(startsWith)`	Compare column by column
31	Valid parentheses	STACK: push openers, pop on closers	(same)
32	Sum of numbers in a string	`match(/\d+/g)` → `map(Number)` → sum	Build numbers digit by digit

How to practise: cover the code, read the **Q**, write version A, then version B without any built-in methods, and run the file to check.

---

10. Quick reference tables

Which methods change the original?

⚠ Mutate (change the original)	✅ Safe (return something new)
`push`, `pop`, `shift`, `unshift`	`[...arr, x]`, `[x, ...arr]`
`splice`	`slice`, `toSpliced`
`sort`	`toSorted`, `[...arr].sort()`
`reverse`	`toReversed`
`fill`	`with(index, value)`
`Object.assign(target, …)`	`{ ...obj, key: value }`
`delete obj.key`	`const { key, ...rest } = obj`

What does it return?

Returns	Methods
New array	`map`, `filter`, `slice`, `concat`, `flat`, `flatMap`, `toSorted`, `toReversed`, `toSpliced`, `with`
One item or `undefined`	`find`, `findLast`, `at`, `shift`, `pop`
Index or `-1`	`indexOf`, `findIndex`, `findLastIndex`
`true` / `false`	`some`, `every`, `includes`, `Array.isArray`, `Object.hasOwn`
Anything (you decide)	`reduce`, `reduceRight`
Removed items (array)	`splice`
New length (number)	`push`, `unshift`
`undefined`	`forEach`

Callback parameters

Method	Callback receives	Extra argument
forEach, map, filter, some, every, find, findIndex	`(element, index?, array?)`	`thisArg?`
reduce, reduceRight	`(accumulator, element, index?, array?)`	`initialValue?`
sort, toSorted	`(a, b)`: return negative, 0 or positive	n/a

Common traps

Trap	Fix
`[10, 9, 1, 2].sort()` → `[1, 10, 2, 9]`	`sort((a, b) => a - b)`
`['1','2','3'].map(parseInt)` → `[1, NaN, NaN]`	`map((s) => parseInt(s, 10))`
`map((n) => { n * 2 })` → all `undefined`	add `return`, or remove the braces
`n => { value: n }` returns undefined	`n => ({ value: n })`
`[].reduce(fn)` throws TypeError	always pass an initial value
`[].every(fn)` → `true`	check `arr.length` first if needed
`{ ...obj }` shares nested objects	`structuredClone(obj)`
`{a:1} === {a:1}` → false	compare contents (deep equal)
`count || 10` replaces `0`	use `count ?? 10`
`b = a` doesn't copy an array	`b = [...a]`
`s.trim()` didn't change `s`	strings are immutable: `s = s.trim()`
`'qa qa'.replace('qa', 'x')` → `'x qa'`	`replaceAll`, or a regex with `/g`
`s.match(/x/g).length` crashes when nothing matches	`(s.match(/x/g) || []).length`
`'a b c'.split(' ')` has an empty item	`split(/\s+/)`
`'5' + 3` → `'53'`	convert first: `Number('5') + 3`
`'B' < 'a'` is true	`localeCompare` for alphabetical order

11. Suggested study plan

Day	Do
1	Read **array-methods-cheatsheet.js** sections 0–4; run it
2	Sections 5–11; then **array-methods-exercises.js** 1–10
3	Exercises 11–20; compare with the solutions
4	**arrays-and-objects-cheatsheet.js** Part A; exercises 1–8
5	Part B; exercises 9–16
6	Part C; exercises 17–22
7	**string-methods-cheatsheet.js**: sections 1–6
8	Sections 7–12
9+	**string-programs.js**: 4 programs a day, writing BOTH the built-in and the loop version
then	**js-interview-programs.js**: 3–5 programs a day. Cover the code, read only **Q** and **Remember**, explain out loud using R-A-C-E-S, write it, run to check

Before an interview: reread the "Remember" columns in sections 7 and 9, and the trap tables in section 10.

12. Troubleshooting

Problem	Fix
`node` is not recognised	Install Node.js from nodejs.org, then reopen VS Code
`Cannot find module`	You're in the wrong folder: run `cd practice` first
`toSorted is not a function`	Node is older than 20; update Node
`Object.groupBy is not a function`	Node is older than 21; update Node, or use the `reduce` version
An exercise shows `✗`	Compare the `expected` and `got` lines; check the cheat sheet section for that method
You want to start an exercise again	Put `return todo();` back in that function
`replaceAll is not a function`	Node is older than 15; update Node, or use `replace` with a `/g` regex